

How web servers talk to applications

Nginx, Node.js, PHP-FPM, and the wiring in between.

Ahmed Shamim Hassan

megamindslearning.com

2026 EDITION

Roles, not programs

Here's the thing that confuses almost everyone at first: “**web server**” and “**application server**” sound like two products you install. They are not products at all. They are roles, meaning jobs that a piece of software can take on. One program can play both roles at once, two programs can split the work between them, and where the line falls depends entirely on the stack you happen to be running.

THE MENTAL MODEL

The **web server** handles everything HTTP-facing. The **application runtime** executes your code. Sometimes they are two separate processes, sometimes they are one process (Apache with `mod_php` runs PHP inside the web server itself), and sometimes a translator protocol sits between them.

HTTP-FACING WORK

- Terminating TLS (the “S” in HTTPS)
- Serving static files
- Compression and caching
- Routing requests to the right place
- Managing thousands of connections

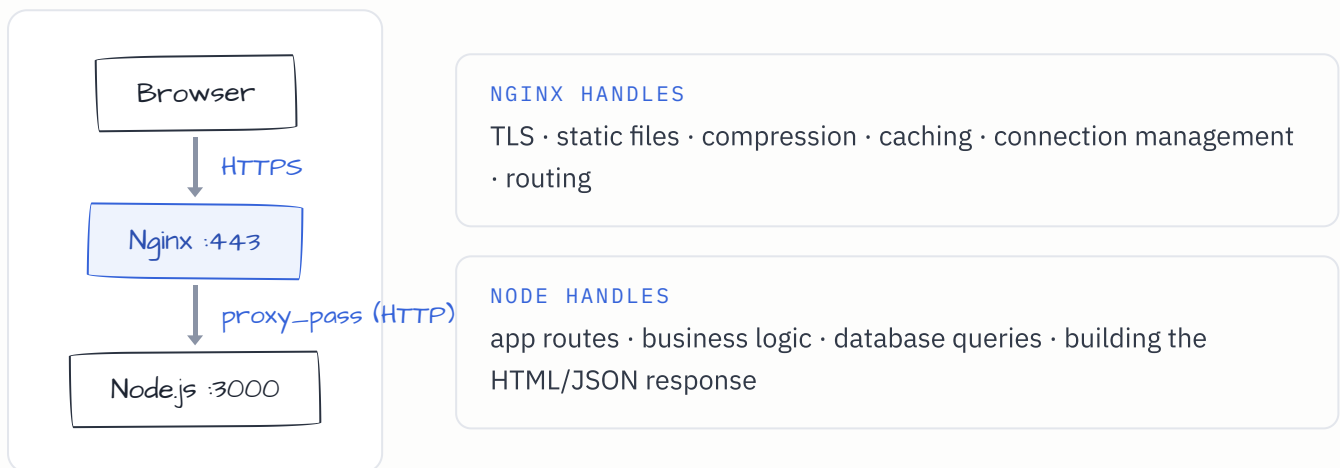
APPLICATION WORK

- Business logic
- Application-level routes
- Talking to the database
- Building the HTML or JSON response
- Everything that makes your app *yours*

Keep one question in mind for the rest of this book: **who receives the request, and how does it reach my code?** Every setup you will ever meet, whether it involves Nginx, PHP-FPM, Gunicorn or plain Node, is simply a different answer to that one question.

Nginx + Node.js: the HTTP handoff

This is the most common pairing you will meet, with Nginx sitting in front and Node.js sitting behind it. Both of them speak HTTP, so the handoff is simple: Nginx receives the request and forwards it to Node, which is what people mean when they say it *proxies* the request. That is why Nginx in this position is called a **reverse proxy**, because it accepts requests on behalf of another server.



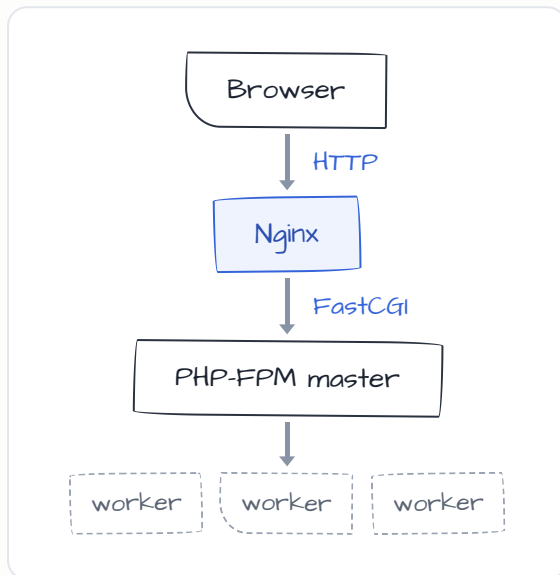
nginx.conf

```
server {  
  listen 443 ssl;  
  location / {  
    proxy_pass http://localhost:3000;  
    proxy_http_version 1.1;  
    proxy_set_header Host $host;  
  }  
}
```

So in this shape there really *is* an application server behind the web server, and that application server is a long-running Node process which understands HTTP all by itself. The two extra lines matter in practice, because Nginx proxies with HTTP/1.0 by default and does not pass the original Host header on its own. Hold on to that detail, because the next chapter breaks it.

Nginx + PHP-FPM: the FastCGI bridge

PHP works differently, and this is where the “web server → application server” picture falls apart. Behind Nginx there's **no HTTP server at all**. What sits there instead is PHP-FPM, a *process manager* that speaks **FastCGI**, which is a protocol built for exactly this kind of handoff.



Nginx matches the request against a location block, decides it is PHP's job, and forwards it over FastCGI together with the script path and the usual CGI variables. PHP-FPM hands it to an idle worker, the worker runs `index.php`, and the result travels back the same way.

The PHP process never listens on a public HTTP port, which is why the Nginx config points at a *socket* instead of a URL.

```
# PHP: a socket, not a URL
fastcgi_pass unix:/run/php/php-fpm.sock;

# compare with Node:
proxy_pass http://localhost:3000;
```

COMMON MIX-UP

PHP-FPM is not “the PHP version of Node.” Node is one long-running app that understands HTTP. PHP-FPM keeps a pool of worker processes alive and talks FastCGI, and each request runs your script from a clean state inside one of those workers. They occupy the same seat in the diagram, but they are very different creatures.

Four shapes of a backend

There is no single true architecture called “web server → application server.” There are at least four common shapes, and all of them are legitimate.

A Just files



A static site. Nginx serves the files itself. No application server anywhere.

B Proxy to an HTTP app



The backend speaks HTTP and Nginx just forwards. Node, Gunicorn or Uvicorn on the Python side, and Laravel Octane all live here.

C Bridge protocol



The thing behind Nginx does not speak HTTP at all. A bridge protocol such as FastCGI, SCGI or uwsgi carries the request to the runtime.

D No front door



The app listens on a public port itself, with no Nginx in front. This is perfectly valid, and uvicorn with FastAPI or Kestrel with ASP.NET often run this way too.

Next time someone shows you a stack, don't ask “where's the application server?” Ask **which of these shapes it is**, and everything else falls into place.

The naming mess

Why is this topic so confusing? Because we name software after whichever role it happens to be playing at the moment. The same program collects three or four labels, and none of them are wrong.

SOFTWARE	YOU'LL HEAR IT CALLED	WHAT IT ACTUALLY IS
Nginx	“web server”, “reverse proxy”, “load balancer”	An HTTP server that can serve files, forward requests, and balance load. Each label describes one use.
Node.js	“runtime”, “HTTP server”, “app server”	A JavaScript runtime built on V8. It only becomes a server when your code opens a listening socket.
PHP-FPM	“PHP's app server”	A FastCGI process manager for PHP workers. Not an HTTP server at all.

server.js

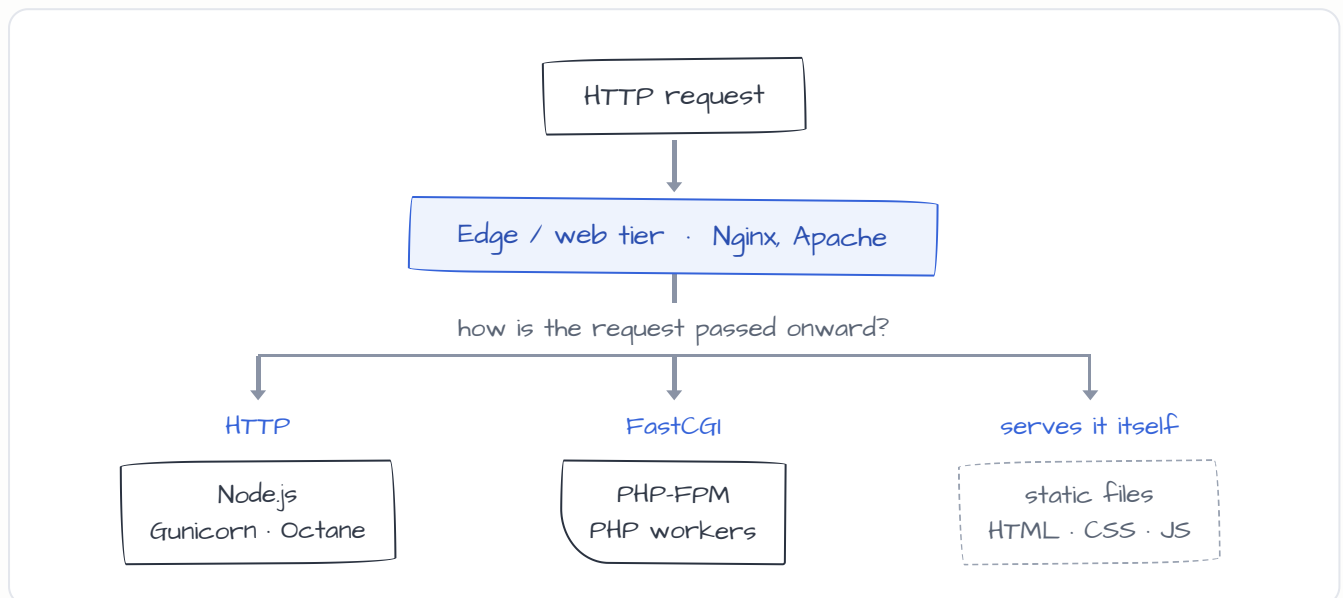
```
// Node.js is just a runtime...
const http = require('http');

// ...until this line turns it into an HTTP server
http.createServer(handler).listen(3000);
```

So “is Node a web server or an application server?” is the wrong question. It's a runtime playing whichever role your code gave it. Likewise, PHP itself is not an application server, and PHP-FPM is simply the mechanism that lets a web server reach PHP worker processes.

The question that matters

Instead of memorizing stacks, ask two things about any setup: who receives the HTTP request, and how does that process pass it onward?



Nginx → HTTP → Laravel Octane

Octane keeps Laravel loaded in memory and accepts requests itself, so it really is an application server.

Nginx → FastCGI → PHP-FPM

This is not an HTTP application server, because Nginx is speaking FastCGI to a process manager.

THE QUESTION THAT MATTERS

Who receives the HTTP request, and how does that process communicate with the code that actually runs the application? Answer that, and any stack diagram becomes readable.

The whole book on one page

1. “Web server” and “application server” are **roles, not products**.
2. Nginx + Node: two HTTP speakers. Nginx reverse-proxies to a long-running app.
3. Nginx + PHP-FPM: FastCGI to a **process manager**, so there is no HTTP server behind the curtain.
4. There's no one true architecture: files, HTTP proxy, bridge protocol, or direct exposure all work.
5. The useful question: **who gets the request, and how does it reach the code?**

SETUP	HANDOFF	BEHIND THE EDGE
Static site	none	Nothing. Nginx serves the files itself.
Nginx + Node.js	HTTP · proxy_pass	A long-running Node process on :3000.
Nginx + Gunicorn	HTTP	Python worker processes running your WSGI or ASGI app.
Nginx + PHP-FPM	FastCGI · socket	A process manager and its PHP workers.
Node / uvicorn alone	HTTP · direct	The app is the front door.

reverse proxy accepts requests on another server's behalf.

TLS termination decrypting HTTPS at the edge.

runtime executes code, and is not a server until you make it one.

FastCGI a protocol for handing requests to a non-HTTP process.

process manager keeps a pool of workers alive and feeds them requests.

edge / web tier whatever receives the raw HTTP request first.

MEGAMINDS LEARNING

Learn backend properly, not in fragments

Topics like this one, in this much depth, are what we work through together in the Backend Cohort.

10

WEEKS

20

LIVE CLASSES

1

PRIVATE DISCORD

[Join the Backend Cohort](#)

backendcohort.ahmedshamim.com

Ahmed Shamim Hassan · 2026